Development of an Interactive Graphical Interface for Real-Time Visualization of the Goertzel Algorithm

Fernanda S. Bucheri, Gabriel Reale M. de Oliveira, André M. de Oliveira
Federal University of São Paulo
São José dos Campos, SP, Brazil

***Abstract*—This article presents the development and implementation of an interactive graphical interface for real-time visualization of the Goertzel Algorithm, with a primary focus on the design and construction of the frontend. The system includes a backend responsible for audio capture and execution of the Goertzel algorithm, along a frontend architecture developed in C++. The interface, built using the ImGui and ImPlot libraries, dynamically displays the processed information and provides tools for configuring multiple frequency analyzers, selecting the audio source, and monitoring results in real time through visualizations such as magnitude-over-time curves and a general spectrogram. The results demonstrate the effectiveness of the proposed interface, highlighting a responsive, clear, and user-friendly solution for spectral analysis.**

***Index Terms*—Goertzel algorithm, spectral analysis, real-time visualization, digital signal processing, ImGui, ImPlot, frequency analysis, C++.**

I. Introduction

Digital Signal Processing is fundamental in various areas of Engineering and Computer Science. Spectral analysis, in particular, is crucial for understanding the frequency composition of a signal [1], and traditionally, the Fast Fourier Transform (FFT) is the standard tool for this analysis [2], [5]. However, in scenarios where interest is restricted to a limited number of frequencies, the FFT may introduce unnecessary computational overhead [5].

The **Goertzel Algorithm** is a Digital Signal Processing method recognized for its efficiency in detecting specific frequencies, being a low-computational-cost alternative to the FFT in spectral monitoring scenarios [4], [5].

Thus, this article focuses on the development of an **interactive frontend interface** for the real-time visualization of the Goertzel Algorithm execution. The central objective of this article is to detail the architecture and implementation of the frontend, which utilizes the *ImGui* and *ImPlot* libraries to transform the algorithm's numerical output into intuitive graphical representations, allowing the user to configure and monitor multiple frequency analyzers simultaneously.

The focus is on the software solution and the visualization technologies employed.

II. Foundations and Technologies Used

In this section, we describe the Goertzel Algorithm and the technologies employed for the construction of the interface.

A. The Goertzel Algorithm

The Goertzel Algorithm is a Digital Signal Processing method used to efficiently compute individual components of the Discrete Fourier Transform (DFT) of a discrete-time signal. Instead of evaluating the full spectrum, as in the FFT, the Goertzel algorithm targets specific frequency bins, making it particularly suitable when only a small set of frequencies is of interest, see [4], [5]. Its main advantage lies in computational efficiency when the number of frequencies of interest is small, making it ideal for spectral monitoring applications [5]. It operates as a second-order recursive digital filter, designed to efficiently calculate a single component of the Discrete Fourier Transform (DFT) of a discrete signal $x[n]$ [5].

The implementation of the Goertzel algorithm can be described by a recurrence equation equivalent to a resonant filter tuned to the frequency of interest [4], [5]. For an input signal $x[n]$ of length L, the algorithm computes the DFT component associated with the normalized angular frequency

$$\Omega_k = \frac{2\pi k}{L} \tag{1}$$

where $k \in 0, \ldots, L-1$. The corresponding physical frequency is given by $f_k = \frac{k}{L} f_s$, where f_s is the sampling rate. In particular, the recurrence equation is

$$s[n] = x[n] - 2\cos(\Omega)\, s[n-1] + s[n-2], \tag{2}$$

where $s[n]$ represents the internal state of the filter at instant n, with initial conditions $s[-1] = 0$ and $s[-2] = 0$. After processing the L samples, the magnitude of the spectral component at the frequency of interest can be obtained directly from the last two states of the recurrence, as per

$$|X(\Omega)|^2 = (s[L-1] - \cos(\Omega)s[L-2])^2 + (\sin(\Omega)s[L-2])^2, \tag{3}$$

eliminating the need for operations with complex numbers during the recursive stage and concentrating the magnitude calculation at the end of processing [4]. It is important to point out that the algorithm evaluates discrete frequency bins, and therefore its resolution is limited by the frame length L. Since the analysis is performed over finite-length signal frames, which implicitly corresponds to the use of a rectangular window, then the signal may contain frequency components that do not exactly match the analyzed frequency bin [2], [5],

which is exactly frequency leakage. This effect can influence the magnitude estimation and should be considered when interpreting the results, especially in real-time visualization scenarios.

B. Graphical User Interface Technologies

The development of the interactive graphical interface is the central focus of this work. For this, several libraries were used:

- **ImGui (Immediate Mode GUI):** A lightweight library for creating graphical interfaces that uses the *Immediate Mode* paradigm. In this model, the interface is not stored internally by the library; instead, all visual elements, such as buttons, text, and controls, are redrawn at each rendering frame. This contrasts with traditional interfaces, which maintain components and their states permanently. The immediate mode makes the interface behavior simpler and more predictable, besides facilitating synchronization with rapidly updating data, being especially suitable for interactive applications and real-time visualization, such as in monitoring the results of the Goertzel algorithm. The library documentation can be found in [3].
- **ImPlot:** An extension of ImGui aimed at generating scientific and engineering plots. In the project, it was used to display various graphs, such as magnitude over time and the spectrogram *(heatmap)*, enabling continuous and clear visualization of the results obtained by the algorithm [6].
- **GLFW and OpenGL:** Libraries responsible for creating the application window and providing the graphical context necessary for rendering the elements produced by ImGui and ImPlot [7], [8].

III. System Architecture

The system was designed with a modular architecture, in which the responsibilities of signal processing *(backend)* and visualization *(frontend)* are clearly and independently separated, as illustrated in Fig. 1. This organization favors scalability, maintenance, and extensibility of the system.

IV. Implementation Analysis

The complete code is available in the project repository [9].

The architecture evidences the data flow from the capture of the acoustic signal to its analysis and subsequent visualization, being structured into three main layers: audio input, processing backend (responsible for managing the analyzers and executing the Goertzel Algorithm, developed by another collaborator), and user interface (focus of this work).

A. Audio Input

Signal acquisition begins at the audio input, responsible for capturing ambient sounds. The microphone or another system audio source provides a digital signal, represented by a sequence of discrete samples obtained at a specific sampling rate. This rate defines both the temporal resolution of the signal and the range of frequencies that can be accurately represented.

The availability of audio sources and the configuration of the sampling rate are performed by the *PulseAudio* library.

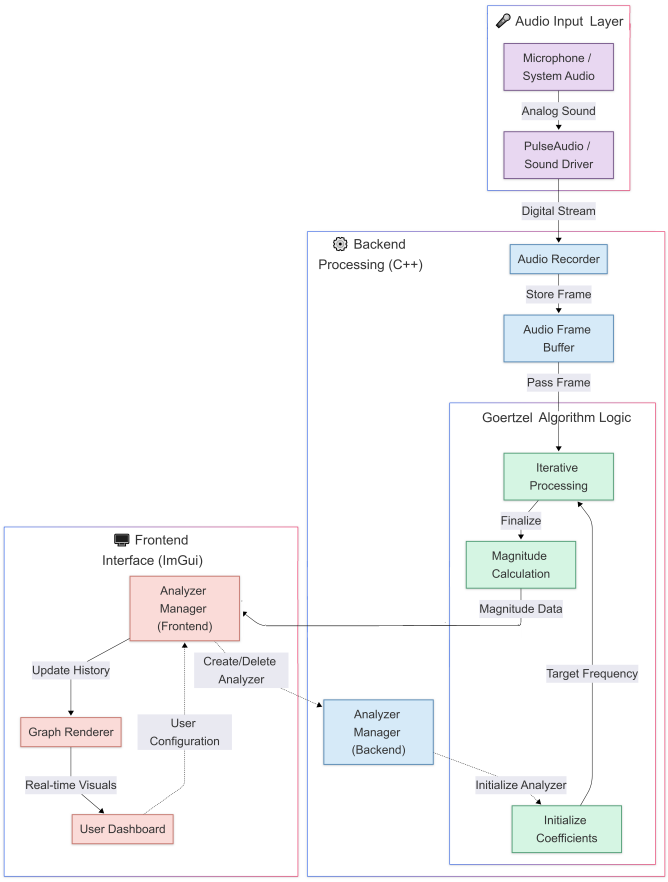

Fig. 1. System block diagram

B. Backend: Signal Processing

The backend constitutes the computational core of the system, being responsible for audio signal acquisition, data organization, and spectral processing.

Signal capture is performed through a component of type `Recorder`, which continuously obtains digital samples from the audio system. These samples are organized into fixed-size blocks (frames), stored in a buffer, ensuring stability in the data flow for processing.

```
Recorder<BUFFER_SIZE> BackEnd::recorder("");
array<float,BUFFER_SIZE> BackEnd::frame;

void BackEnd::update(){
    recorder.record(frame);
}
```

Listing 1. Audio capture and buffer storage (BackEnd.cpp)

Spectral processing is performed using the Goertzel algorithm, implemented in the `Goertzel` class. This algorithm acts as a digital filter optimized for detecting specific frequencies.

Before processing, each analyzer is configured with the frequency of interest by calculating the filter coefficients, ensuring that the frequency is within the *Nyquist* limit. The backend supports the simultaneous execution of multiple analyzers, organized in a mapping structure that associates each frequency with an instance of the Goertzel algorithm, allowing for the dynamic creation or removal of analyzers as needed.

Communication with the frontend occurs primarily through the `queryFrequency` function, which executes processing for a specific frequency and returns its normalized magnitude.

```
float BackEnd::queryFrequency(float frequency){
    auto analyzer = analyzers.find(frequency);
    if (analyzer != analyzers.end()){
        float magnitude = (analyzer->second).
            execute(frame);
        normalization = normalization > magnitude
             ? normalization * decay : magnitude;
        return magnitude / normalization;
    }
    else
        return -1;
}
```

Listing 2. Frequency magnitude query (BackEnd.cpp)

To improve visualization stability, a dynamic normalization mechanism is employed. The variable `normalization` follows a peak magnitude value with a decay factor applied over time. When a newly computed magnitude exceeds the current normalization value, it becomes the new reference; otherwise, the normalization value decays gradually. This approach prevents abrupt scaling changes while preserving responsiveness to significant signal variations.

As a result, the returned magnitude is expressed relative to a slowly varying reference, enhancing the readability of real-time plots. However, this normalization does not preserve absolute amplitude information and should be interpreted accordingly.

C. Frontend: Interface and Control

The *frontend* corresponds to the layer responsible for direct user interaction. It is through it that the system is configured, frequency analyzers are controlled, and processing results are visualized in real time. Thus, the frontend acts as a bridge between the user and the *backend*, translating user actions into commands and presenting data in an understandable way.

Furthermore, the system maintains control of the available audio sources and the frequency the user wishes to monitor. From this information, the frontend coordinates the entire application usage cycle: initializes the graphical environment, allows for the creation and removal of analyzers, controls their execution, and organizes the data display in different forms of visualization.

In general, the frontend was designed to simplify interaction with the system, hiding technical processing details and offering an intuitive interface. While the backend focuses on calculations and signal processing, the frontend is dedicated to making these results accessible, clear, and easy to interpret.

The `GoertzelAnalyzerData` structure is used to store specific data for each frequency analyzer monitored by the frontend. It contains the target frequency (`frequency`), a status indicator (`running`), a magnitude history (`spectrum_history`), and a timestamp for update rate control (`last_update_time`). Thus, each analyzer has its own state in the frontend, allowing for continuous updates and organized data visualization.

D. Frontend Code Analysis

The implementation of the *frontend* is detailed below, focusing on key functions that configure the interface and coordinate communication with the *backend*.

1) Interface Initialization (`initialize`): The function `FrontEnd::Application::initialize` is responsible for configuring the graphical environment and establishing initial communication with the *backend*. In it, ImGui and ImPlot contexts are created, along with integration with GLFW and OpenGL.

In addition to the graphical configuration, the function queries the *backend* to obtain the list of available audio sources via `BackEnd::querySources()`. If sources are detected, the first one is set as default with `BackEnd::setSource()`, allowing the system to start audio capture and processing right upon opening the interface.

```
void FrontEnd::Application::initialize(GLFWwindow
    * window) {
    IMGUI_CHECKVERSION();
    ImGui::CreateContext();
    ImPlot::CreateContext();
    ImGuiIO& io = ImGui::GetIO(); (void)io;
    io.ConfigFlags |=
        ImGuiConfigFlags_NavEnableKeyboard;
    io.ConfigFlags |=
        ImGuiConfigFlags_NavEnableGamepad;

    ImGui::StyleColorsDark();
    ImPlot::GetStyle().Colormap =
        ImPlotColormap_Plasma;

    ImGui_ImplGlfw_InitForOpenGL(window, true);
    ImGui_ImplOpenGL3_Init("#version 130");

    s_audio_sources = BackEnd::querySources();
    if (s_audio_sources.empty()) {
        s_audio_sources.push_back("Nenhuma fonte
            de áudio disponível");
    }

    if (!s_audio_sources.empty()) {
        BackEnd::setSource(s_audio_sources[
        s_selected_source_index]);
    }
}
```

Listing 3. Frontend initialization (FrontEnd.cpp)

2) Main Rendering Loop (`render`): The function `FrontEnd::Application::render` constitutes the core of the interface update cycle, being responsible for the continuous rendering of the main window and the organization of functionalities into tabs. In each frame, the frontend prepares the new ImGui *frame*, creates the main window, and distributes content among interface sections.

In the main window, functionalities are organized into three tabs: *Settings*, *Visualization per Analyzer*, and *Overall Visualization*. At the end of each cycle, ImGui renders the elements on the screen, synchronizing the interface with the data updated by the *backend*.

```
void FrontEnd::Application::render(GLFWwindow*
    window) {
    ImGui_ImplOpenGL3_NewFrame();
    ImGui_ImplGlfw_NewFrame();
    ImGui::NewFrame();

```

```
    ImGui::SetNextWindowPos(ImVec2(0, 0));
    ImGui::SetNextWindowSize(ImGui::GetIO().
        DisplaySize);
    ImGui::Begin("Goertzel Analyzer", nullptr,
        ImGuiWindowFlags_NoDecoration |
        ImGuiWindowFlags_NoResize |
        ImGuiWindowFlags_NoMove);

    ImGui::BeginChild(
        "ScrollableRegion",
        ImVec2(0, 0),
        false,
        ImGuiWindowFlags_AlwaysVerticalScrollbar
            |
        ImGuiWindowFlags_AlwaysHorizontalScrollbar

    );

    if (ImGui::BeginTabBar("##MainTabBar")) {
        if (ImGui::BeginTabItem("Configurações"))
             {
            showConfigurationPage();
            ImGui::EndTabItem();
        }
        if (ImGui::BeginTabItem("Visualização
            Geral")) {
            showOverallVisualizationPage();
            ImGui::EndTabItem();
        }
        if (ImGui::BeginTabItem("Visualização por
             Analisador")) {
            showVisualizationPage();
            ImGui::EndTabItem();
        }
        ImGui::EndTabBar();
    }

    ImGui::EndChild();
    ImGui::End();
    ImGui::Render();
    int display_w, display_h;
    glfwGetFramebufferSize(window, &display_w, &
        display_h);
    glViewport(0, 0, display_w, display_h);
    glClearColor(0.45f, 0.55f, 0.60f, 1.00f);
    glClear(GL_COLOR_BUFFER_BIT);
    ImGui_ImplOpenGL3_RenderDrawData(ImGui::
        GetDrawData());
}
```

Listing 4. Main rendering loop (FrontEnd.cpp)

3) Analyzer Management: The frontend offers specific functions to manage analyzers, allowing for adding, removing, starting, and pausing frequency monitoring. When adding a new analyzer, the frequency provided by the user is first normalized to two decimal places, avoiding duplications caused by small numerical differences. Then, the frontend checks if an analyzer associated with that frequency already exists. If not, the backend is instructed to create the new analyzer via `BackEnd::createAnalyzer()`, and a new entry is stored in `s_analyzers_data`.

This same pattern is followed by the `removeAnalyzer`, `startAnalyzer`, and `stopAnalyzer` functions, which keep the frontend's local structure synchronized with the backend's internal state.

```
void FrontEnd::Application::addAnalyzer(float
    frequency) {
    float freq_to_add = limitToTwoDecimals(
        frequency);
    if (s_analyzers_data.find(freq_to_add) ==
        s_analyzers_data.end()) {
        BackEnd::createAnalyzer(freq_to_add);
        GoertzelAnalyzerData new_data;
        new_data.frequency = freq_to_add;
        s_analyzers_data[freq_to_add] = new_data;
    }
}
```

Listing 5. Adding an analyzer (FrontEnd.cpp)

4) "Settings" Tab (`showConfigurationPage`): In the *Settings* tab, the user can choose the audio source and manage frequency analyzers. The interface provides an input field for entering the desired frequency, as well as buttons to add, pause, or remove analyzers.

Upon pressing the *Add* button, the frontend calls `addAnalyzer` to register the new frequency and `startAnalyzer` to immediately start its processing. Thus, analyzer creation and monitoring initiation are integrated into a single user action, making the interaction more direct and intuitive.

```
    ImGui::Text("Adicionar/Remover Analisador");
    ImGui::TextColored(ImVec4(0.7f, 0.7f, 0.7f,
        1.0f), "Clique sobre o valor para editar
        e digite a frequencia desejada, ou use os
         botões + / -");
    ImGui::InputFloat("Frequencia (Hz)", \&
        s_new_frequency_input, 1.0f, 100.0f, "%.2
        f Hz");
    s_new_frequency_input = limitToTwoDecimals(
        s_new_frequency_input);

    ImGui::SameLine();
    if (ImGui::Button("Adicionar")) {
        addAnalyzer(s_new_frequency_input);
        startAnalyzer(s_new_frequency_input);
}
```

Listing 6. Settings and analyzer creation (FrontEnd.cpp)

5) "Visualization per Analyzer" Tab (`showVisualizationPage`): In this tab, each active analyzer is displayed through a graph showing frequency magnitude over time. The frontend periodically queries the backend for the latest value of the monitored frequency. If the analyzer is running, the new magnitude is added to the history stored in `spectrum_history`. To avoid indefinite growth of the structure, older values are discarded when the sample limit is reached.

This history is then plotted with `ImPlot::PlotLine`, allowing for visual tracking of the analyzer's response evolution over time. Thus, the user can identify variations in the signal clearly and continuously.

```
// ... inside the loop for each analyzer ...
if (data.running && (current_time - data.
    last_update_time > 0)) {
        float magnitude = BackEnd::queryFrequency
            (freq);
        if (data.spectrum_history.size() > 200)
            data.spectrum_history.erase(data.
                spectrum_history.begin());
        data.spectrum_history.push_back(magnitude
            );
        data.last_update_time = current_time;
    }

// ... ImPlot::BeginPlot configuration ...
ImPlot::PushStyleColor(ImPlotCol_Line, IM_COL32
    (255, 100, 100, 255));
```

```
ImPlot::PushStyleVar(ImPlotStyleVar_LineWeight, 2.0f);
ImPlot::PlotLine("Magnitude", x_indices.data(), data.spectrum_history.data(), data.spectrum_history.size());
ImPlot::PopStyleVar();
ImPlot::PopStyleColor();
```

Listing 7. Visualization per analyzer (FrontEnd.cpp)

6) "Overall Visualization" Tab (`showOverallVisualizationPage`): The *Overall Visualization* tab presents a consolidated view of the system, gathering data from all analyzers into a single spectrogram. For this, the frontend organizes magnitude histories into a one-dimensional matrix (`heatmap_data`), where each row represents a frequency and each column represents a point in time. Since analyzers may be registered in any order, frequencies are sorted before plotting, ensuring consistency in vertical axis reading.

After data preparation, the spectrogram is drawn with `ImPlot::PlotHeatmap`. Axes are configured to represent time and frequencies, and Y-axis ticks are customized to display the real values of monitored frequencies. The color scale is adjusted via minimum and maximum magnitude limits, allowing for more precise analysis of signal variations.

```
// ... inside showOverallVisualizationPage ...
int N_freqs = s_analyzers_data.size();
int N_history = 200;

std::vector<float> heatmap_data(N_freqs * N_history);
std::vector<float> freqs_y;

int i = 0;
for (auto& pair : s_analyzers_data) {
    float freq = pair.first;
    GoertzelAnalyzerData& data = pair.second;

    freqs_y.push_back(freq);

    for (int j = 0; j < N_history; ++j) {
        int hist_idx = j - (N_history - data.spectrum_history.size());

        float magnitude = 0.0f;
        if (hist_idx >= 0 && hist_idx < data.spectrum_history.size()) {
            magnitude = data.spectrum_history[hist_idx];
        }

        float magnitude_linear = magnitude;
        heatmap_data[i * N_history + j] = magnitude_linear;
    }
    i++;
}

std::vector<std::pair<float, int>> freq_indices;
for (int k = 0; k < N_freqs; ++k) {
    freq_indices.push_back({freqs_y[k], k});
}

std::sort(freq_indices.begin(), freq_indices.end(), [](const auto& a, const auto& b) {
    return a.first < b.first;
});

std::vector<float> sorted_heatmap_data(N_freqs * N_history);
std::vector<float> sorted_freqs_y;

for (int k = 0; k < N_freqs; ++k) {
    int original_index = freq_indices[k].second;
    float freq = freq_indices[k].first;

    std::copy(
        heatmap_data.begin() + original_index * N_history,
        heatmap_data.begin() + (original_index + 1) * N_history,
        sorted_heatmap_data.begin() + k * N_history
    );

    sorted_freqs_y.push_back(freq);
}
```

Listing 8. Data preparation for the spectrogram (FrontEnd.cpp)

```
// ... inside showOverallVisualizationPage ...
if (ImPlot::BeginPlot("##Spectrogram", ImVec2(w, plot_height), ImPlotFlags_NoMouseText)) {

ImPlot::SetupAxes("Tempo (Amostras)", "Frequencia (Hz)");
ImPlot::SetupAxisLimits(ImAxis_X1, 0, (double)N_history, ImGuiCond_Always);
ImPlot::SetupAxisLimits(ImAxis_Y1, 0, (double)N_freqs, ImGuiCond_Always);

std::vector<double> y_ticks_pos(N_freqs);
std::vector<std::string> y_ticks_labels_str(N_freqs);

for (int k = 0; k < N_freqs; ++k) {
    y_ticks_pos[k] = (double)k + 0.5;
    std::stringstream ss;
    ss << std::fixed << std::setprecision(2) << sorted_freqs_y[k];
    y_ticks_labels_str[k] = ss.str();
}

std::vector<const char*> y_ticks_labels_c_str = convert_to_c_str_array(y_ticks_labels_str);

ImPlot::SetupAxisTicks(ImAxis_Y1, y_ticks_pos.data(), N_freqs, y_ticks_labels_c_str.data());

ImPlot::PlotHeatmap(
    "##SpectrogramData",
    sorted_heatmap_data.data(),
    N_freqs,
    N_history,
    s_min_magnitude, s_max_magnitude,
    nullptr,
    ImPlotPoint(0, N_freqs),
    ImPlotPoint(N_history, 0)
);
ImPlot::EndPlot();
}
```

Listing 9. Spectrogram plotting (FrontEnd.cpp)

V. Results

In this section, results are presented through screenshots of the developed interface. Three frequency analyzers (700 Hz, 1000 Hz, and 1500 Hz) were configured to demonstrate the system's ability to monitor multiple spectral components simultaneously. Figures 2 and 3 illustrate the complete flow of configuration, overall visualization, and individual inspection of a frequency analyzer.

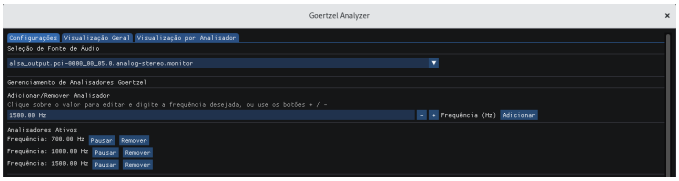

Fig. 2. ”Settings” tab, showing audio source selection and activation of analyzers corresponding to 700 Hz, 1000 Hz, and 1500 Hz frequencies.

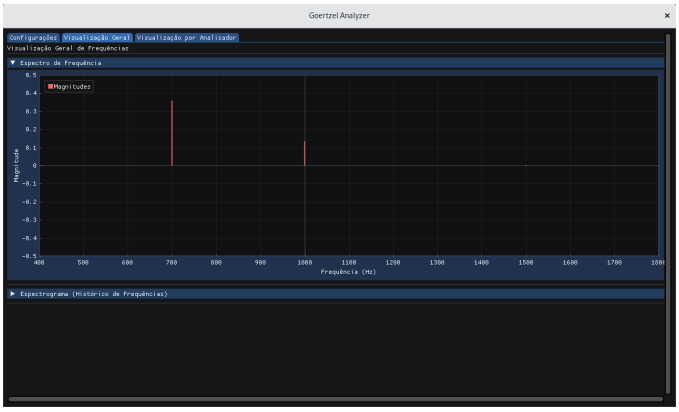

Fig. 3. ”Overall Visualization” tab, displaying the real-time frequency spectrum for analyzed signals.

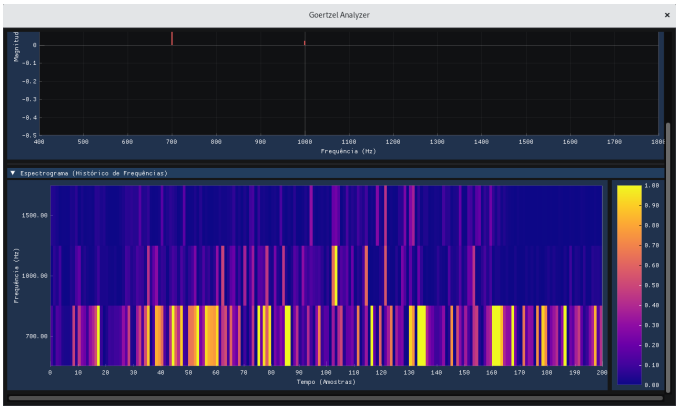

Fig. 4. ”Overall Visualization” tab, presenting the spectrogram (heatmap) showing the temporal history of 700 Hz, 1000 Hz, and 1500 Hz frequencies.

A. Results Discussion

The visualization of magnitude over time (Figure 5) allows for immediate identification of the presence or absence of the **target frequency**. The spectrogram (Figure 4) offers a historical and comparative view, being a powerful tool for frequency pattern analysis. Interface interactivity, enabled by ImGui/ImPlot, allows the user to adjust visualization parameters (such as the *heatmap* magnitude range), facilitating data interpretation. They illustrate the feasibility of using the Goertzel Algorithm in conjunction with a modern graphical interface for real-time spectral analysis.

The system demonstrates responsive behavior suitable for interactive use, with continuous updates of the visualizations as new audio frames are processed. The perceived latency is primarily determined by the frame size used for processing,

Fig. 5. ”Visualization per Analyzer” tab, highlighting the 1000 Hz frequency analyzer and displaying its magnitude over time graph.

and also the audio sampling rate: they define the update interval of the interface. It is important to point out that while no formal latency measurements were made, the implementation proved adequate for real-time monitoring in practical usage scenarios.

VI. Conclusion

The project resulted in the development of an effective and interactive tool for frequency analysis using the *Goertzel Algorithm*. The graphical interface, built with *ImGui* and *ImPlot*, fulfills the objective of making data inspection and interpretation more accessible and visually rich.

The main contribution of this project is the development of an integrated tool that combines a well-established frequency analysis method, the Goertzel algorithm, with a modern and interactive visualization interface. The result is a practical and accessible system for exploring spectral information in real time, particularly in applications where monitoring specific frequency bins is sufficient.

References

[1] A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2010.
[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.
[3] O. Cornut, “Dear ImGui: Immediate Mode GUI,” [Online]. Available: https://github.com/ocornut/imgui. Accessed: Mar. 3, 2026.
[4] G. Goertzel, “An algorithm for the evaluation of finite trigonometric series,” *The American Mathematical Monthly*, vol. 65, no. 1, pp. 34–35, Jan. 1958.
[5] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 3rd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 1996.
[6] E. Pezent, “ImPlot: Immediate Mode Plotting,” [Online]. Available: https://github.com/epezent/implot. Accessed: Mar. 3, 2026.
[7] GLFW Developers, “GLFW: Simple library for creating windows, contexts and surfaces,” [Online]. Available: https://www.glfw.org/. Accessed: Mar. 5, 2026.
[8] The Khronos Group, “OpenGL: The industry standard for high performance graphics,” [Online]. Available: https://www.opengl.org/. Accessed: Mar. 5, 2026.
[9] G. Reale, “TCC project repository,” [Online]. Available: https://github.com/g-reale/TCC. Accessed: Mar. 5, 2026.